Vercel setup & backend CORS guide

Follow these steps to make the frontend work on Vercel when your backend is at `http://51.102.152.208:8080`.

1) Preferred: Use Next proxying (recommended)

- In Vercel project Settings → Environment Variables add:
  - `BACKEND_INTERNAL_BASE_URL` = `http://51.102.152.208:8080`
  - Apply to: Production (and Preview if desired)
- Redeploy the project (push to `main` or click "Redeploy").
- Why: Next will rewrite `/api/:path*` to `${BACKEND_INTERNAL_BASE_URL}/api/:path*` and the browser will only talk to your Vercel origin (no CORS).

2) Alternate: Direct client calls

- If you prefer browser → backend direct calls, set in Vercel:
  - `NEXT_PUBLIC_API_BASE` = `http://51.102.152.208:8080/api`
- Redeploy.
- Note: Backend must allow your Vercel origin via CORS and set cookies accordingly.

3) Backend CORS (required for direct client calls)

Example Express middleware (dynamic origin allow):

```
const allowed = new Set([
  'https://aishortlist-app.vercel.app',
  'https://ai-screen-ing.netlify.app'
]);

app.use((req, res, next) => {
  const origin = req.headers.origin;
  if (origin && allowed.has(origin)) {
    res.setHeader('Access-Control-Allow-Origin', origin);
    res.setHeader('Access-Control-Allow-Credentials', 'true');
    res.setHeader('Access-Control-Allow-Methods',
'GET,POST,PUT,PATCH,DELETE,OPTIONS');
    res.setHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization,
X-Requested-With');
  }
  if (req.method === 'OPTIONS') return res.sendStatus(204);
  next();
});

// When setting cookies:
res.cookie('refreshToken', token, {
  httpOnly: true,
```

```
  secure: true,
  sameSite: 'None',
  path: '/',
});
```

Fastify CORS example (plugin):

```
fastify.register(require('@fastify/cors'), {
  origin: (origin, cb) => {
    const allowed = ['https://aishortlist-app.vercel.app', 'https://ai-screen-
ing.netlify.app'];
    if (!origin || allowed.includes(origin)) cb(null, true);
    else cb(new Error('Not allowed'), false);
  },
  credentials: true,
});
```

Flask (Python) example using `flask-cors`:

```
from flask_cors import CORS
app = Flask(__name__)
CORS(app, origins=["https://aishortlist-app.vercel.app", "https://ai-screen-
ing.netlify.app"], supports_credentials=True)
```

Spring Boot (Java) example:

```
@Bean
public WebMvcConfigurer corsConfigurer() {
    return new WebMvcConfigurer() {
        @Override
        public void addCorsMappings(CorsRegistry registry) {
            registry.addMapping("/api/**")
                .allowedOrigins("https://aishortlist-app.vercel.app",
"https://ai-screen-ing.netlify.app")
                .allowedMethods("GET","POST","PUT","PATCH","DELETE","OPTIONS")
                .allowCredentials(true);
        }
    };
}
```

4) Verify after deploy

From your machine (or CI), run:

```
```

```
# Test Vercel proxy (after setting BACKEND_INTERNAL_BASE_URL + redeploy)
curl -i -X POST https://aishortlist-app.vercel.app/api/auth/login -d '{}' -H
"Content-Type: application/json"

# Test backend direct with Origin header (CORS response)
curl -i -X OPTIONS http://51.102.152.208:8080/api/auth/login \
  -H "Origin: https://aishortlist-app.vercel.app" \
  -H "Access-Control-Request-Method: POST"

curl -i -X POST http://51.102.152.208:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -H "Origin: https://aishortlist-app.vercel.app" \
  -d '{}'
```

If Vercel returns 404 but backend returns 200, the rewrite is not applied — ensure `BACKEND_INTERNAL_BASE_URL` was set and the project redeployed.

5) If you want, I can prepare a small verification script inside the repo to run these checks. Run `node scripts/verify_endpoints.js <url>` to test.

If you want me to add more backend framework examples or push a small verification script, tell me which framework and I'll add it.